Build Steps

Part 1 of 5 · Set up your agent in the new builder

1 Create your agent

Open Copilot Studio (copilotstudio.microsoft.com), select **Create**, then **New agent**. In the redesigned builder everything lives under the **Build** tab, with a settings panel down the right. Set:

Agent name Audit & Reporting Q&A Copilot
Description Answers questions on month-end actuals, journal variances, control exceptions, and audit trail for cost-centre owners and internal auditors.
Solution / Environment Finance Solution · Default environment
Icon Use a recognisable colour and emoji, or upload a brand icon

2 Choose your model

Open the **Model** selector at the top of the right-hand panel. Copilot Studio now lets you pick the model your agent reasons with. This build uses **Claude Sonnet 4.6**; you can switch later without rebuilding.

Reasoning is built in

- The agent reasons over your instructions, knowledge, skills and tools, and decides what to do each turn
- Handles paraphrases, multi-step asks and follow-ups
- Blends a knowledge lookup with a tool call in one reply
- Calls the right skill when intent matches

Choosing a model

- Claude Sonnet 4.6: strong reasoning, a good default here
- Lighter models: lower latency and cost for simple, high-volume flows
- Switch the model and re-run the same evaluation set to compare
- Check data residency and model availability for your tenant

In regulated finance, treat model choice as a control decision: record which model the agent runs, check data residency for your tenant, and re-run your evaluation set whenever the model changes.

3 Write your instructions

The **Instructions** box is the heart of the Build tab. Define the agent's role, audiences and rules here. The agent reasons over these together with everything in the right-hand panel. Paste:

You are the Audit & Reporting Q&A Copilot. You help two audiences:
1) Cost centre owners reviewing their journals.
2) Internal auditors reviewing the control exceptions queue.

Rules:
- Use the uploaded journal dataset as your single source of truth.
- Always cite the JournalID for every number you quote.
- Never invent or round numbers; quote them as recorded.
- Flag rows with ControlFlag = Fail at the top of any answer.
- For actions (flag a journal for review), call the tool. Do not just say it is flagged.
- Owners only see their own cost centre rows; auditors see everything.

Build Steps

Part 2 of 5 · Ground it in your data

4 Add Knowledge

Open **Knowledge** in the panel, select **Add knowledge**, then **File**, and upload **Finance_Audit_Data.xlsx**. Knowledge gives the agent trusted context to guide its answers. This file contains:

- Journal data (journal ID, period, cost centre, account, description)
- Amounts (debit USD, credit USD, net USD)
- Control signals (control flag: Pass / Review / Fail, variance vs budget %)
- Audit trail (approver, posted date)

Layer more sources later: SharePoint sites, Dataverse, Microsoft Graph connectors, or public web.

5 Bring in Microsoft IQ

Open **Microsoft IQ** in the panel. Microsoft IQ brings in work context, business data and app signals from across your tenant, so the agent grounds answers in more than the file you uploaded.

For this agent, Microsoft IQ resolves the auditor or owner identity and links each journal to its source document in SharePoint, so a flagged row can be traced to evidence without leaving the chat. Microsoft IQ honours each user's existing permissions, so the agent only surfaces what the signed-in user is already allowed to see.

Build Steps

Part 3 of 5 · Give it skills and tools

6 Add a Skill

Open **Skills** in the panel, then **Add a skill**. A skill defines a reusable behaviour through structured instructions. It is the new home for the conversational flows you used to build as topics: the agent calls the skill when a user's intent matches.

Skill name: Investigate Control Exception
Description: Walks an auditor through the rows currently flagged Fail or Review, explains the control rule that fired, and offers to formally raise the exception for follow-up.
When to use it (intent hints the agent reasons over):

- Show me control exceptions for this period
- What journals failed control
- Investigate the Fail rows
- Walk me through control issues

Structured steps:

- Ask for the period (default to the most recent in data)
- Filter ControlFlag in (Fail, Review), order by absolute NetUSD descending
- For each row, cite JournalID, CostCentre, Account, NetUSD, VarianceVsBudgetPct
- Explain the likely control rule that fired (for example variance over 10%)
- Ask which JournalID to formally flag, if any
- Call 'Flag Journal for Audit Review' with the chosen JournalID and reason

7 Add a Tool

Open **Tools**, then **Add a tool**. **Skills decide what to do; tools take the action.** A tool is a typed, callable action with named inputs and outputs that the model fills from the conversation. Choose one of: a **prebuilt connector** (Outlook, Teams, SharePoint, Dataverse, ServiceNow and more), a **Power Automate cloud flow**, a **custom HTTP request**, or a **Model Context Protocol (MCP) server**. This build uses a Power Automate flow because it does three things in one call: file the finding, notify the owner and audit lead, and return a finding ID.

Tool name: Flag Journal for Audit Review
Type: Power Automate flow to a SharePoint list 'Audit Findings', plus a Teams adaptive card
Description (the model reads this to decide when to call): Files a formal audit finding against a specific journal row and notifies the cost-centre owner and the internal audit lead.
Inputs the agent collects or infers:

- journalId — text, from knowledge
- period — text, e.g. '2026-04'
- findingCategory — text, one of Variance / Control / Documentation / Other
- auditorNote — text, the rationale in the auditor's own words
- suggestedAction — text, one of Reclass / Reverse / Document / No action

Outputs returned to the agent:

- findingId — text, e.g. AF-20260608-0117
- status — text, e.g. Filed
- cardPosted — yes/no

7.1 · Build the cloud flow that does the work

From inside Copilot Studio choose **Tools > Add a tool > New > Power Automate**. This opens the flow designer pre-wired with the Copilot Studio trigger and, crucially, keeps the flow in the **same environment** as your agent (Finance Solution · Default). An environment mismatch is the most common reason a finished tool never appears in the picker, so confirm the environment switcher (top-right) before you build.

Trigger — *When an agent calls the flow*. Add five text input parameters, named exactly: `journalId`, `period`, `findingCategory`, `auditorNote`, `suggestedAction`.

– **Action 1 – generate the finding ID.** Add *Initialize variable* `findingId` = `concat('AF-', formatDateTime(utcNow(),'yyyyMMdd'), '-', rand(100,999))`. (Or let SharePoint / Dataverse return the ID instead.)
– **Action 2 – file the finding.** SharePoint *Create item* (or Dataverse *Add a new row*) into the **Audit Findings** list. Map: Title = findingId, JournalID = journalId, Period = period, Category = findingCategory, Note = auditorNote, Action = suggestedAction, Status = 'Filed'.
– **Action 3 – notify the stakeholders.** Microsoft Teams *Post card in a chat or channel* to the cost-centre owner and the internal audit lead, using an adaptive card that shows the finding ID, JournalID, period, category and note.
– **Final action – return to the agent.** Add *Respond to the agent* (Return value(s) to Copilot Studio) with outputs findingId = `variables('findingId')`, status = 'Filed', cardPosted = true.
– **Save** the flow as `Finance – Flag Journal for Audit Review`.

7.2 · Register and map the tool in your agent

– Back in Copilot Studio the flow is attached automatically; if you built it separately, **Tools > Add a tool > Flows** and select it.
– Set the display name to **Flag Journal for Audit Review** and write the **Description** for the model — it is what the agent reads to decide when to call the tool.
– For each input set **Fill using**: journalId → *dynamically* from the row the skill is investigating; period, findingCategory, auditorNote and suggestedAction → **Fill with AI**, and add an input description so the model extracts the right value (for suggestedAction, list Reclass/Reverse/Document/No action and what each means).
– Confirm `findingId` is returned to the agent so the skill can read it back — satisfying your instruction rule “do not just say it is flagged”.
– **Connection & auth:** set the SharePoint and Teams connections the flow uses. The connectors chosen here are exactly what your DLP policy must permit (see Step 10).

7.3 · Worked example — what flows in and out

When an auditor says *"flag J-2271 for review, undocumented vendor accrual"*, the model fills the inputs and calls the tool with this payload:

```
{
  "journalId":       "J-2271",
  "period":          "2026-04",
  "findingCategory": "Documentation",
  "auditorNote":     "Undocumented vendor accrual; evidence missing",
  "suggestedAction": "Document"
}
```

The flow files the row, posts the card, and returns:

```
{
  "findingId":  "AF-20260608-0117",
  "status":     "Filed",
  "cardPosted": true
}
```

The agent then replies in natural language, quoting the real finding ID: *"I've filed audit finding AF-20260608-0117 against journal J-2271 (Documentation) and notified the cost-centre owner and the audit lead."*

7.4 · Test the tool before you trust it

– **In isolation:** open the flow in Power Automate, choose *Test* > *Manually*, supply the five inputs, run it, and confirm a row appears in the Audit Findings list, a card posts to the owner and audit lead, and the run returns all three outputs.
– **End to end:** in **Preview** (Step 11) type "flag J-2271 for review, evidence missing" and check the activity view shows Flag Journal for Audit Review firing with the inputs the model inferred and the finding ID it read back.

Build Steps

Part 4 of 5 · Extend, remember and govern

8 Add Connected agents

Open **Connected agents**. A connected agent lets this agent collaborate with others, delegating specialist work and sharing context across them (agent-to-agent). Keep each agent focused on what it does best.

Connect a **Document Evidence Agent** that fetches supporting invoices and contracts from SharePoint for a flagged journal, so a finding arrives with its evidence already attached.

9 Turn on Memory

Toggle **Memory** on. Memory lets the agent remember interactions, workflows and context across sessions, so returning users do not have to repeat themselves. For this agent it remembers:

- The periods under review, so the agent opens on the right close
- Prior findings the auditor raised, so duplicates are surfaced not re-filed

Memory is scoped to each user and follows your retention and DLP policy. Review what the agent retains before you publish.

10 Govern your agent

Open the agent settings and lock these down before real users arrive:

Authentication: use *Authenticate with Microsoft (Entra)* so the agent runs as the signed-in user and inherits their permissions.
Content moderation: set the moderation level (start at Medium) and review it for your audience.
Data loss prevention: confirm your DLP policy covers the connectors your tools use — for Flag Journal for Audit Review that means the SharePoint and Teams connectors must sit in the same DLP group, or the tool is blocked at runtime.
Channels and access: review which channels are enabled and who can use the agent.

Build Steps

Part 5 of 5 · Preview, evaluate and publish

11 Preview your agent

Switch to the **Preview** tab to chat with your agent as you build. Open the **activity view** on any response to inspect:

- Which skill, tool or knowledge source fired, and in what order
- The inputs the agent collected and what each tool returned
- Citations linking each answer back to a row of your data
- Latency per step, useful when tuning multi-tool flows

12 Evaluate

Open the **Evaluate** tab, then **New evaluation**. Build a test set so you can measure quality before and after every change. Score each response on **groundedness** (did it stay inside the knowledge), **relevance** (did it address the question), and **accuracy** (did it match the expected outcome).

Test set: Audit & Reporting Copilot · Baseline v1
Input: Show all journals with ControlFlag = Fail in period 2026-04
Expect: Returns rows filtered by Period and ControlFlag = Fail, cites JournalIDs.
Input: Which cost centres had a variance over 10% vs budget?
Expect: Returns rows where VarianceVsBudgetPct is over 10, ordered descending, with NetUSD cited.
Input: Flag journal J-2271 for audit review, undocumented vendor accrual
Expect: Triggers the Investigate Control Exception skill and calls Flag Journal for Audit Review.

13 Publish and Monitor

Click **Publish**, then enable the channels your users live in: Microsoft Teams, Microsoft 365 Copilot, a custom website embed, or other channels. Teams and M365 Copilot need tenant admin approval, so plan for that lead time. After publishing, open the **Monitor** tab to watch sessions, resolution and escalation outcomes, and quality trends over time, and to see where users drop off.

Try These Live

Sample queries to test your agent during the build-along.

Control Exceptions
Show all journals flagged Fail in the current period. · Which cost centres have the most Review flags this period?

Variance Lookups
What is the largest debit posted to cost centre CC-440 this period? · Show journals with variance over 10% vs budget, biggest absolute value first.

Governed Action
Flag journal J-2271 for review, auditor note 'evidence missing'.
Try a paraphrase: 'this entry looks off, raise it with audit'.

How to Think About This Agent

This agent replaces	With
• Manual pivots over the GL extract • Email chains to chase exception explanations • Audit findings logged in unstructured spreadsheets	✓ Citations linking every number to a JournalID ✓ Structured audit findings flowing into SharePoint ✓ Faster close, with control exceptions surfaced first

Trusted, governed, enterprise AI.
Built once, used everywhere.

Tip for real-world use
Bind the agent to your authenticated identity via Entra so a cost-centre owner only ever sees their own rows. Run the evaluation set after every period close to confirm the agent still answers the top-20 control questions exactly the same way.

You've built a Copilot Studio agent ✓
You now have an AI agent that can:

- Cite every number to a journal row
- Flag control exceptions formally
- Surface variance and control risk first
- Speed up the audit trail